```
import pandas as pd
import numpy as np
import seaborn as sns
import scipy.stats as stats
import matplotlib.pyplot as plt
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
```

Load Data & Check for Copycats (EDA & VIF)

```
# 1. Load the dataset
df = pd.read_csv("/content/marketing_sales_data.csv.csv")
df = df.dropna()  # Clean missing values safely

# 2. Separate our independent variables to check for multicollinearity
X_features = df[['TV', 'Radio', 'Social Media']]

# Convert categorical 'TV' column to numerical using one-hot encoding and ensure integer type
X_features = pd.get_dummies(X_features, columns=['TV'], drop_first=True).astype(int)

# Add a constant column for the VIF calculation matrix
X_vif = sm.add_constant(X_features, has_constant='add')

# 3. Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data["Feature"] = X_features.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1]) if X_vif.columns[i] != 'const']

print("--- MULTICOLLINEARITY CHECK (VIF) ---")
print(vif_data)
```

```
--- MULTICOLLINEARITY CHECK (VIF) ---
        Feature       VIF
0         Radio  3.463224
1  Social Media  1.646863
2        TV_Low  4.072879
3     TV_Medium  2.221533
```

Build the Multiple Linear Regression Model

```
# Define dependent variable (Y) and independent features (X)
Y = df['Sales']
X = X_features # Use the preprocessed X_features with one-hot encoded 'TV' column

# Add the constant intercept
X = sm.add_constant(X)

# Fit the Multivariate OLS model
multi_model = sm.OLS(Y, X).fit()

# View the comprehensive performance breakdown
print(multi_model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.904
Method:                 Least Squares   F-statistic:                     1340.
Date:                Mon, 15 Jun 2026   Prob (F-statistic):          3.08e-287
Time:                        13:56:21   Log-Likelihood:                -2713.1
No. Observations:                 572   AIC:                             5436.
Df Residuals:                     567   BIC:                             5458.
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          219.7180      6.167     35.630      0.000     207.606     231.830
Radio            3.0098      0.233     12.894      0.000       2.551       3.468
Social Media    -0.2213      0.672     -0.329      0.742      -1.541       1.098
TV_Low        -153.9883      4.931    -31.228      0.000    -163.674    -144.303
TV_Medium      -75.1405      3.626    -20.724      0.000     -82.262     -68.019
==============================================================================
Omnibus:                       60.195   Durbin-Watson:                   1.874
```

```
Prob(Omnibus):                  0.000   Jarque-Bera (JB):                17.925
Skew:                           0.046   Prob(JB):                     0.000128
Kurtosis:                       2.138   Cond. No.                         139.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Diagnostic Plots (Advanced Evaluation)

```
residuals = multi_model.resid

plt.figure(figsize=(12, 5))

# Q-Q Plot for Normality
plt.subplot(1, 2, 1)
stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Multi-Model Q-Q Plot")

# Residual vs Fitted Plot for Homoscedasticity
plt.subplot(1, 2, 2)
plt.scatter(multi_model.fittedvalues, residuals, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")
plt.title("Multi-Model Residual Plot")

plt.tight_layout()
plt.show()
```